

- 1) Student justifies the type of Sagemaker instance they created: I have chosen ml.t3.medium instance for creating notebook instance. This instance type has 2 vCPUs and 4 GiB memory and priced at \$0.05/per hour which is the lowest amongst all instance types under notebook instance category. [Machine Learning Service – Amazon SageMaker Pricing – AWS](#) . In previous exercises, I have found that this instance type provides good performance considering most of the heavy-lifting is delegated to separate instances suitable for running training and inference job. So, it's a good ROI instance type. Even if you forget and keep it running say for 10 hours, still the cost is minimal \$0.5.
- 2) Create and open an EC2 instance: for EC2 instance, I chose ml.g4dn.xlarge instance type. This instance type has 4 vCPUs, 16Gib of memory, 1 NVIDIA T4 GPU With 16 GB memory and is charged at \$0.736/hour and seems quite economical given the size of the project. This is the same instance I have been using in training deep learning models in previous projects. So long as you keep batch size limited to 64, this machine provides very fast training times.
- 3) EC2 Code: I find EC2 code quite similar to the one provided in hpo.py which is used to train the model using Pytorch Estimator objects. Just that it has values hard-coded and can be run on standalone mode just by using command: "python <filename>.py". If we can scriptify it using argparse module and add "__name__ == __main__" block, then we can very well invoke it using command line options like how sagemaker training jobs invoke the training scripts.
- 4) Lambda Function: lambda function is performing the same work as Predictor object does on deployed end point in sagemaker. First, it initializes a variable that refers to endpoint. Next it creates a runtime sagemaker session. Using the runtime it invokes the endpoint supplying endpoint name, data(contained in event variable. This may be image data or text data that needs to be classified) and few other parameters. This is the same task as predictor.predict performs in sagemaker environment. Next, it reads the response that it got from endpoint in json format which is returned along with status code and other header elements to the calling program.
- 5) Lambda Function Deploy result: Result is contained in another file "lambdaoutput.txt" submitted as part of this project. Jupyter notebook has same information as well.
- 6) The writeup contains a description of potential vulnerabilities in the IAM setup, including the following: roles that have "FullAccess" policies attached, roles that are old or inactive, and roles with policies for functions that the project is no longer using:
As it looks like the permission granted to lambda function role (FullSageMakerAccess) is excessive in my opinion. If we can find out the exact permissions required to get the job done, we should be using that instead. Next, our setup can be secured further by using VPC(Virtual Private Clouds) to restrict access. VPCs are similar to subnets on a local area network. Any role which is old, we should find out what purpose it is serving and if it is required at all under current circumstances. Roles which are inactive and no longer be used should be cleaned up. In my opinion setting up a security audit to be performed periodically (like 3 months) should be a good approach prior to taking up cleanup task.

- 7) The writeup clearly describes traffic, cost, and efficiency considerations for both concurrency and auto-scaling.

Concurrency : Concurrency refers to how many incoming client request lambda function can serve at one point of time. When requests are coming from internet, along with API gateway, lambda function is the first hop before it goes to endpoint. Lambda function actually is responsible for invoking model endpoint.

I have configured 3 reserved concurrency for lambda function. This will determine the maximum level of concurrency that lambda function can achieve at anytime. Further, I have configured 2 provisioned concurrency. This means that we will always have two instances running to serve the client requests which can be further scaled up to 3 should the client load goes up. The configured values are appropriate given that this is just a test environment. We should also keep in mind that provisioned concurrency should be configured carefully as it is an “always on” instance and incurs cost.

AutoScaling is configured at endpoint level. This refers to how many incoming connections can be served by endpoint simultaneously. If there are more incoming connections, endpoint instances should be scaled up accordingly. Rather than monitoring and configuring this manually we can setup autoscaling that makes decision based on threshold number of incoming connections which when breached it will spin up a new endpoint instance and scale down when not needed as per defined cool down time defined as part of the configuration. Leaving the instances up for long time when the traffic load is low, incurs additional cost which can be avoided. Further having too few instances serving high traffic load will increase latency experienced at client end and may cause their workflow to fail or slowdown and may lead to customer dissatisfaction. So it is very important the scale in and scale down operations are performed efficiently. In our case since this is just test environment, I have configured 1 instance as the desired value which can scale upto two instances should the traffic load goes up.

As the normal procedure goes, these values should be configured and tested thoroughly in a production like environment(UAT or pre-prod) before these are applied to actual production environment.